Lecture_7

2025-10-08

Credit: **Dr. Preeti Jagadev**

Mapping in R

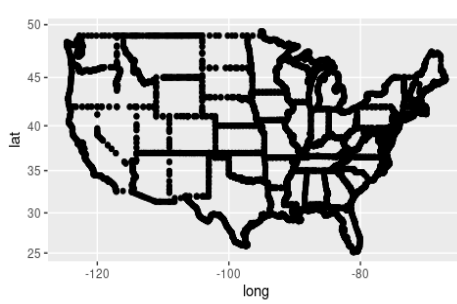
```
install.packages("maps")
install.packages("ggmap")

library(ggplot2)
library(maps)
states <- map_data("state")
head(states)
```

```
##        long      lat group order  region subregion
## 1 -87.46201 30.38968     1     1 alabama      <NA>
## 2 -87.48493 30.37249     1     2 alabama      <NA>
## 3 -87.52503 30.37249     1     3 alabama      <NA>
## 4 -87.53076 30.33239     1     4 alabama      <NA>
## 5 -87.57087 30.32665     1     5 alabama      <NA>
## 6 -87.58806 30.32665     1     6 alabama      <NA>
```

Explanation of the output:

- **group**: A unique identifier for the set of points that form a single polygon (like a state or an island belonging to a state). This tells the program which points to connect to draw a continuous shape.
- **order:** The sequence in which the points must be connected to draw the outline correctly.
- **region:** The name of the state (e.g., “new york,” “california”).

```
ggplot(states)  + geom_point(aes(x=long,y=lat)) + coord_map()
```



Preeti Jagadev

Explanation of the code:

This code is used to draw the map of the states by plotting all the individual coordinates that make up their outlines. Here's a detailed, step-by-step breakdown of what each piece does:

1. **ggplot(states)**: This starts the entire plot-making process and specifies the data to be used.

2. **geom_point(aes(x=long, y=lat))**: This tells ggplot2 how to draw the data.

3. **geom_point():** Specifies the geometric object to use, in this case, points (dots).

4. **aes(x=long, y=lat):** Specifies the Aesthetics, how the data variables map to the visual elements. It says to use the long column for the horizontal position (x-axis) and the lat column for the vertical position (y-axis).

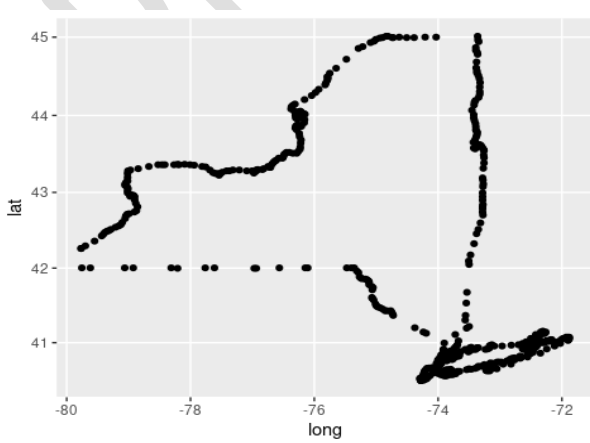
5. **coord_map()**: This applies a map projection to the plot.

   Simple Meaning: "Treat the plot as a map and apply a proper projection to account for the Earth's spherical shape."

Without coord_map(), the map would look distorted and squashed because the raw latitude and longitude coordinates are simply plotted on a flat, rectangular plane, which doesn't correctly represent the true distances and shapes of the states. coord_map() fixes this distortion.

Zoom into NY

```
library(dplyr)

NYData <- states %>% filter(region=="new york")


ggplot(NYData)  +
  geom_point(aes(x=long,y=lat)) + coord_map()
```



Explanation:

This set of R code commands does two main things:

It selects only the data points needed to draw the outline of New York State from your full map dataset, and then it draws those points on a map.

Here is a simple breakdown of each line:

1. **Selecting the Data (The Filter)**

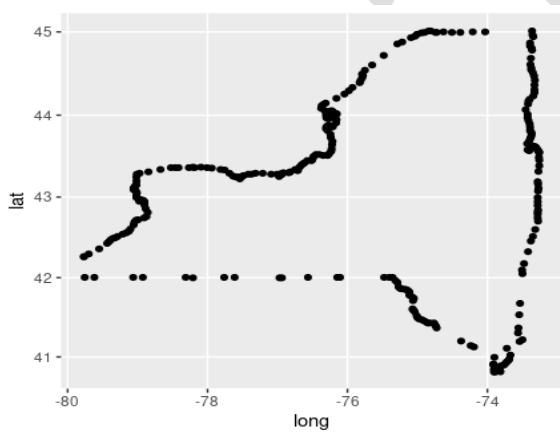
library(dplyr): This loads the dplyr package, which contains useful tools for working with data, most importantly the pipe operator (%>%) and **the filter ()** function.

states %>% filter(...): This takes your entire states data (which contains coordinates for all US states) and pipes it (sends it) into the filter function.

filter(region == "new york"): This command looks at the region column for every coordinate point in the data and keeps only the points where the region name is "new york".

Zoom into a Part of NY:

```
states %>% filter(group==35) %>%
     ggplot()  +
          geom_point(aes(x=long,y=lat)) + coord_map()
```



Explanation:

states %>% filter(group == 35): From the big dataset of all U.S. states, this keeps only the points (coordinates) that belong to one particular state (the one labeled as group 35).Each "group" is basically one state's outline.

Creating a Simple Map

```
map.simple <- ggplot(states) +
     geom_polygon(color="black", fill="white",
                          aes(x=long,y=lat, group=group)) +
     coord_map()
map.simple
```

Creating a DataFrame With Geometry

```
usData <- data.frame(stateName=state.name,  area=state.area)
usData$centerX=state.center$x
usData$centerY=state.center$y

#Make sure everything is lowercase
usData$stateName <- tolower(usData$stateName)

#Combine dataframes using the merge function
usDataWithGeom <- merge(usData,
        states,  by.x="stateName",by.y="region")

#Don't lose the order for the points in polygon
usDataWithGeom <- usDataWithGeom %>%  arrange(order)
```

Explanation:

This R code block prepares a final dataset called **usDataWithGeom** that combines geographical facts (like area and center point) with the actual map coordinates needed to draw the US states.

1. **Create the Facts Table:**

A new data frame (usData) is created to store facts about the states, using built-in R datasets:

stateName: The name of each US state.

area: The land area of each state.

centerX and centerY: The geographical coordinates for the approximate center point of each state.

2. **Prepare the Link Column:**

usData$stateName <- tolower(usData$**stateName)**: The state names in the new usData table are converted to all lowercase. This is a crucial step to ensure they exactly match the lowercase state names found in the states data frame (the one you previously loaded using map_data("state")), allowing the next step to work correctly.

3. **Combine the Tables (The Merge)**

usDataWithGeom <- merge(usData, states, by.x="stateName", by.y="region")

This is the most important step: it combines the two different data frames:

usData (the table of facts like area and center points).

states (the table of coordinates that define the state outlines).

by.x="stateName", by.y="region": This tells the merge function to link the tables based on the common state name column, which is called stateName in the facts table and region in the map coordinates table.

The result is a single, large table called **usDataWithGeom**. Every single coordinate point now has the area and center coordinates attached to it.

4. **Restore the Drawing Order**

usDataWithGeom <- usDataWithGeom %>% arrange(order)

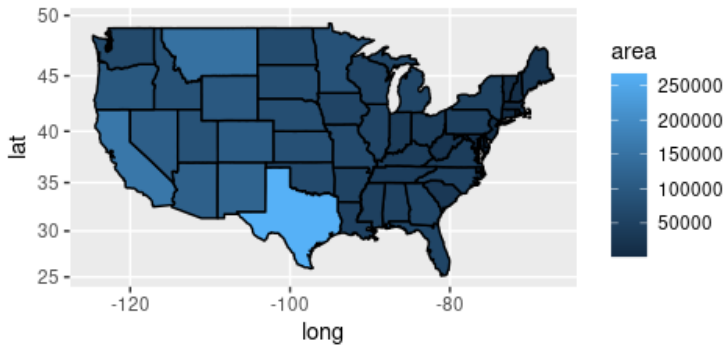
The merge() function often scrambles the original order of the map coordinates.

arrange(order): This command uses the order column (which was in the original states data) to sort the rows back into the exact sequence needed to draw each state's outline correctly, ensuring the map lines don't get jumbled.

Creating a Filled Map (Based on Area):

```
ggplot(usDataWithGeom) +
    geom_polygon(color="black",
                 aes(x=long,y=lat, group=group,  fill=area)) +
    coord_map()
```

Preeti Jagadev

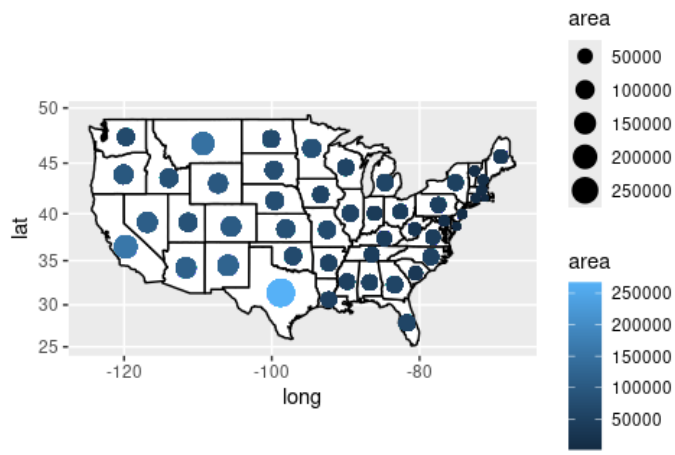


Add a Point to the Map

```
map.simple + geom_point(aes(x = -100, y = 30))
```

Add Points to the Map

```
ggplot(usDataWithGeom) +
  geom_polygon(color="black", fill="white",
               aes(x=long,y=lat, group=group)) +
  geom_point(
      aes(x=centerX,y= centerY, color=area,size=area)) +
  coord_map()
```



Explanation:

group: group makes sure each state is drawn as a separate shape.

geom_point(aes(x=centerX, y=centerY, color=area, size=area)): Adds a point in the middle of each state (using centerX, centerY). The color of the point depends on the value of area. The size of the point also depends on area.

Create an Image (Raster) Map:

```
library(ggmap)

bb <- c(left = min(usDataWithGeom$long),
        bottom = min(usDataWithGeom$lat),
        right = max(usDataWithGeom$long),
        top = max(usDataWithGeom$lat))
register_stadiamaps (key = "f0c40ccd-cd2e-4571-87ed-2700a0e109ad", write =
TRUE)

map <- get_stadiamap (bbox = bb, zoom=4)

ggmap (map)
```

Jagadev

Explanation:

This R code's job is to draw a customized map showing where your data points are located. Think of it like getting directions and then taking a picture:

bb <- c(left = min(...), ... top = max(...)): Figure out the edges of your map. Look at all the locations (long and lat) in your data (usDataWithGeom) and find the absolute western, southern, eastern, and northernmost points. This creates a virtual box (bb) around all your data."

register_stadiamaps(...): Log into the map provider. Use your key (key ="...") to tell the map service (Stadia Maps) that you are allowed to download their maps. The write = TRUE part saves your password so you don't have to enter it again later."

map <- get_stadiamap (bbox = bb, zoom=4): Download the map image. Go to the Stadia Maps website and download the image tile that perfectly fits the box (bbox = bb) you just defined. Use a zoom level of 4, which shows a wide area, like an entire country or large region.

Preeti Jagadev